

规模化快速模型编辑

Eric Mitchell, Charles Lin, Antoine Bosselut, Chelsea Finn, Christopher D. Manning

斯坦福大学

eric.mitchell@cs.stanford.edu

摘要

虽然大型预训练模型在各种下游任务中取得了令人印象深刻的结果，但现有的大型模型仍然会出错，即使是准确的预测也可能随着时间的推移而过时。由于不可能在训练时检测到所有此类故障，因此最好能让此类模型的开发人员和最终用户都能纠正不准确的输出，同时在其他方面保持模型完好无损。然而，大型神经网络所学习的表征具有分布式、黑盒子的性质，因此很难进行这种有针对性的编辑。如果只有一个有问题的输入和新的期望输出，微调方法往往会过度拟合；其他编辑算法要么计算不可行，要么在应用于超大型模型时根本无效。为了实现简单的大规模事后编辑，我们提出了梯度分解模型编辑器网络（MEND），它是一个小型辅助编辑网络集合，使用单个所需的输入输出对，对预训练模型的行为进行快速的局部编辑。MEND 学习如何转换通过标准微调获得的梯度，并使用梯度的低秩分解使这一转换的参数化变得简单易行。即使是 100 亿以上参数的模型，MEND 也能在一天之内在单个 GPU 上完成训练；一旦训练完成，MEND 就能对预训练模型快速应用新的编辑。我们用 T5、GPT、BERT 和 BART 模型进行的实验表明，MEND 是唯一一种能有效编辑参数超过 100 亿的模型行为的模型编辑方法。代码和数据见 <https://sites.google.com/view/mend-editing>。

1 简介

越来越多的大型模型提高了各种现代计算机视觉（Huang 等人，2017 年；Chen 等人，2022 年），尤其是自然语言处理（Vaswani 等人，2017 年；Brown 等人，2020 年）问题的性能。然而，部署和维护此类模型的一个关键挑战是在部署后发布补丁以调整模型行为（Sinitisin 等人，2020 年）。当神经网络产生不良输出时，由于模型表征的分布式特性，针对单个输入或少量输入进行局部更新以纠正其行为并非易事。例如，当提示“谁是英国首相？”时，在 2019 年训练的大型语言模型可能会赋予特蕾莎·梅（Theresa May）比鲍里斯·约翰逊（Boris Johnson）更高的概率。（的提示时，特蕾莎·梅可能会鲍里斯·约翰逊等人比给出更高的概率（真实大型语言模型的例子见表 2；Lazaridou 对 LM 时空泛化失败的系统研究见（2021））。理想的模型编辑

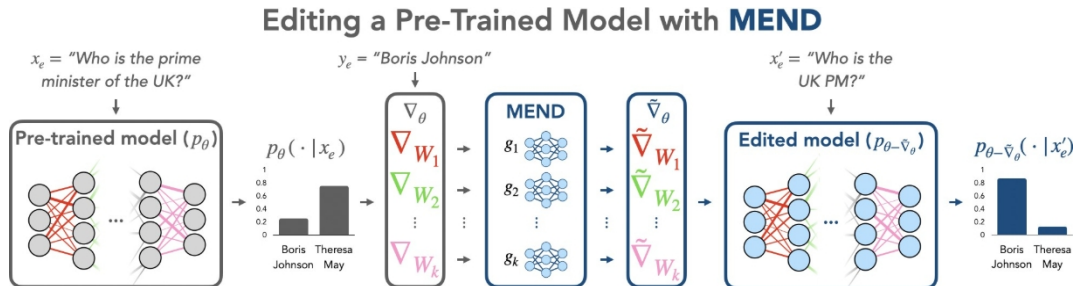


图 1：所提出的 MEND 算法通过训练一组 MLP 来修改模型梯度，以产生局部模型编辑，从而实现可编辑性，这种编辑不会损害模型在无关输入上的性能。正如第 5.1 节所示，MEND 可以高效地训练和应用编辑，即使对于非常大的模型也是

作为会议论文发表于 2022 年国际会议研究中心（
如此
ICLR 2022）。

该程序可以快速更新模型参数，以增加 *鲍里斯·约翰逊* 的相对可能性，而不改变无关输入的模型输出。该程序产生的编辑结果具有 *可靠性*，能成功改变模型对有问题输入的输出（例如，*谁是英国首相？*）；*局部性*，对无关输入的模型输出影响最小（例如，*梅西效力于哪个运动队？*）

进行此类编辑的一个简单方法是在需要修正的单个示例上使用新标签进行额外微调。然而，即使对微调前和微调后参数之间的距离进行限制，对单个示例进行微调也往往会导致过拟合（Zhu 等人，2020 年；De Cao 等人，2021 年）。这种过拟合会导致局部性和一般性的失败。虽然在编辑考试中进行微调，同时在训练集上继续训练，能更好地加强局部性，但我们的实验表明，它仍然缺乏通用性。此外，它需要在测试期间持续访问完整的训练集，对计算要求更高。作为替代方案，最近的研究考虑了学习编辑模型的方法。Sinitisin 等人（2020 年）描述了一种双层元学习目标，该目标能找到一个模型初始化，对该模型进行标准微调后，单个编辑示例就能产生有用的编辑。虽然有效，但学习这种可编辑代表的计算要求使得很难扩展到最需要快速有效编辑的超大模型（见图 3）。De Cao 等人（2021 年）描述了一种基于学习的计算高效的替代方法，但在我们的实验中，它无法编辑非常大的模型。因此，我们设计了一种程序，它能产生可靠、局部和一般的编辑，同时能轻松扩展到拥有 100 多亿个参数的模型。

我们的方法训练轻量级 *模型编辑器网络*，当输入给定校正的标准微调梯度时，就能对预先训练好的模型权重进行编辑，将梯度作为编辑的一个信息丰富的起点（见图 1）。由于梯度是高维对象，直接将梯度映射到新参数更新的函数参数化代价巨大。即使对于单个 $d \times d$ 权重矩阵，天真的实现也需要从 $\mathbb{R}^{(d) \times (d^2)}$ 到 $\mathbb{R}^{(d) \times (d \times (d^2))}$ 的映射，这对于 $d \approx 10^4$ 的大型模型是不切实际的。然而，通过 *将这一梯度分解为秩-1 外积形式*，我们的方法反而能够学习一个函数 $g: \mathbb{R}^{(d) \times (d \times (d^2))} \rightarrow \mathbb{R}^{(d) \times (d \times (d^2))}$ 。我们称这种方法为 *梯度分解模型编辑网络* (MEND)。MEND 将这些梯度映射函数参数化为具有单隐层的 MLP（图 2），与它们编辑的模型相比，只需使用少量参数。MEND 可应用于任何预训练模型，与预训练无关。

这项工作的主要贡献是提出了一种可扩展的快速模型编辑算法，它可以利用微调梯度的低秩结构编辑非常大的预训练语言模型。我们在各种语言相关任务和转换器模型上进行了实证评估，结果表明 MEND 是唯一一种可以持续编辑最大的 GPT 风格（Radford 等人，2019 年；Black 等人，2021 年；Wang 和 Komatsuzaki，2021 年）和 T5（Raffel 等人，2020 年）语言模型的算法。最后，我们的消融实验凸显了 MEND 关键组件的影响，表明 MEND 的变体有可能扩展到拥有数千亿参数的模型。

2 模型编辑问题

模型编辑的目标是使用一对输入 x_e 和期望输出 y_e 来改变 *基础模型* 对 x_e 的输出及其 *等价邻域*（相关的输入/输出对），同时保持模型对无关输入的行为不变（Sinitisin 等人，2020 年；De Cao 等人，2021 年）。对于问题解答模型，*模型编辑器* 会使用问题和新的预期答案来更新模型，更新的方式是正确回答问题及其语义等价的重述，而不影响模型在无关问题上的表现。一些模型编辑器，包括我们的模型编辑器，在应用编辑之前会先经过一个训练阶段（Sinitisin 等人等人，2020 年 2011 年；De Cao，）。等人，2021 年），使用指定编辑类型的编辑训练数据集 D^{tr} 。

更确切地说，基础模型 $f_\theta: X \times \Theta \rightarrow Y$ 是一个可微函数，它将输入 x 和参数集 θ 映射到输出 y 。模型编辑器是一个函数 $E: X \times Y \times L \times \Theta \times \Phi \rightarrow \Theta$ ，它将 *编辑输入* x_e 、*编辑标签* y_e （类标签或词块序列）、损失函数 $l_\phi: X \times Y \times \Theta \rightarrow \mathbb{R}$ 、基本模型参数 θ 和可选的编辑器参数 ϕ 映射到一组新的模型参数 θ_e 。根据过去的工作（De Cao 等，2021 年），我们使用损失函数 $l_\phi(x, y, \theta) = -\log p_\theta(y|x)$ ，但也可以选择其他函数。模型编辑器的评估是在一个保持不变的

数据集 $D^{tr} = \{(x_e, y_e, x_{loc}, x, y_c)_{e,c}\}_{i,j}$ 。对于学习模型编辑器参数 ϕ 的算法，a

数据集 D^{tr} 中包含与 D^{tr} 类似的元组，通常比预先训练的

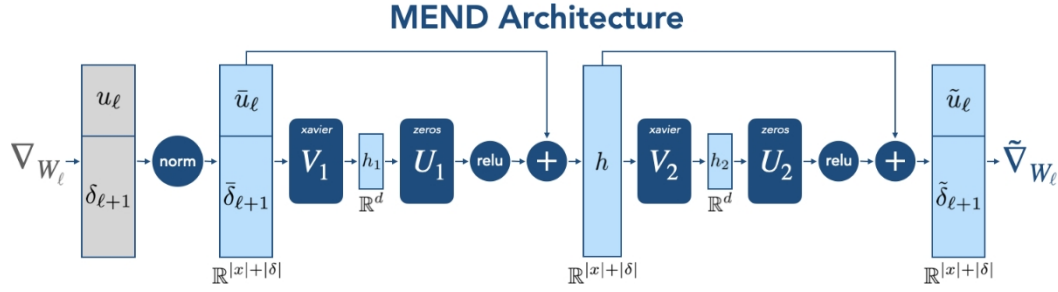


图 2: MEND 架构由两个连续的区域组成, 这两个区域的初始化都是为了计算精确的标识函数。左图 MEND 网络的输入是 $\{ \delta_{(E)}^{(t+1)}, u_{(E)} \}$, 即 rank-1 梯度的分量。右图 MEND 网络会生成一个新的秩-1 更新 $\tilde{\nabla}_{W_l}$, 并将其添加到权重 $W_{\mathcal{E}}$ 中以编辑模型。

模型的原始训练集。局部性输入 x_{loc} 只是一个随机取样的输入, 用于量化模型预测对无关输入的变化程度。替代编辑输入和标签 x' 和 y' 从 $x_{(e)}$ 和 $y_{(e)}$ 的等价邻域 $N(x_{(e)}, y_{(e)})$ 中采样、

是对 $x_{(e)}, y_{(e)}$ 进行编辑后, 编辑模型应泛化到的示例集。对于 $x_{(e)}, y_{(e)} = \text{谁是英国首相? 鲍里斯-约翰逊}$, $N(x_{(e)}, y_{(e)})$ 可能包含 $x', y' =$

英国首相是谁? x_{loc} 可能是 梅西效力于哪支球队?

在这项工作中, 如果编辑后模型能预测编辑输入 x_e 的编辑标签 $y_{(e)}$, 我们就称该模型编辑器为可靠编辑器。

如果编辑前模型和编辑后模型之间存在分歧, 我们称模型编辑器为本地编辑器。

即 $E_{\text{KL}}(p_{\text{KL}}(y|x_{\text{loc}}) \| p_{\text{KL}}(y|x_{(e)}))$ 是很小的。最后, 我们说

如果编辑后模型预测标签 y 的条件是 x , 对于 $(x', y') \in N(x_{(e)}, y_{(e)})$, 如果一个模型编辑器的时间和内存需求为

计算 φ 和评估 E 都很小。我们定义编辑成功率 (ES) 来概括可靠性和通用性。它的衡量标准是编辑后的模型 $p_{(S)}(y)$ 在编辑输入以及从等价邻域中均匀抽取的输入上的平均准确度:

$$\text{ES} = \mathbb{E}_{(x', y') \sim N(x_{(e)}, y_{(e)})} \left(\frac{1}{|N(x_{(e)}, y_{(e)})|} \sum_{(x', y') \in N(x_{(e)}, y_{(e)})} \mathbb{1} \{ \arg \max_y p_{(S)}(y) | x_{(e)} = y_{(e)} \} \right) \quad (1)$$

3 梯度分解模型编辑网络

从广义上讲, MEND 是一种将原始微调梯度转化为更有针对性的参数更新的学习方法, 可在单一步骤中成功编辑模型。MEND 使用 f_{S} 和一个

编辑训练集 D^{edit} 生成一个模型编辑器网络集合 g_A , 它可以编辑模型的

在测试时给定新的编辑对 $(x_{(e)}, y_{(e)})$ 的权重。每个 g_A 都会将特定层 l 的微调梯度转换为该层的参数更新, 从而提供前面所述的可靠性、定位性、通用性和效率特性。由于梯度是高维对象, 这些网络的输入和输出空间也是高维的, 因此以计算可行的方式对其进行参数化是一项挑战。在本节中, 我们将从全连接层梯度的低阶因式分解入手, 介绍 MEND 如何做到这一点。

3.1 高维梯度的参数高效变换

MEND 网络 g_A 的输入是第 l 层的微调梯度 $\nabla_{(W)}(y_{(e)} | x_{(e)}, y_{(e)})$, 输出是该层的参数编辑, 我们称之为 $\tilde{\nabla}_{W_l}$ 。如前所述, 对于 $d \times d$ 权重矩阵, 该函数有 d^2 个输入和输出。即使 g_A 是一个没有隐藏层的线性网络, 并且只产生一个秩-1 的参数编辑 (根据 Hu 等人 (2021 年) 观察到的低秩模型编辑的有效性), 这个函数仍然需要 $d^2(d+d) = 2d^3$ 个参数。对于秩为 r 的 g_A 的低秩线性参数化, 我们需要 $r(d^2 + 2d)$ 个参数, 考虑到某些模型的 $d \approx 10^4$, 这对于非三阶 r 来说仍然是不可接受的代价 (Raffel 等, 2020)。

MEND 利用以下事实解决了这一问题, 即微调梯度 g_A 的输入是一个秩-1 矩阵: 相对于 MLP 第 l 层权重 W_A 的损失梯度 L 是一个秩-1 矩阵

对于每个 B 批元素 $\nabla_{W_l} L = \sum_{i=1}^B \delta^{(l)} u^{(l)}$, 其中 $\delta^{(l)}$ 是批元素 i 相对于第 l 层的预激活+1, 而 $u^{(l)}$ 是第 l 层的输入。

¹ 有关估计 KL-发散的更多详情, 请参阅附录 C.2。

算法 1 MEND 训练

1: **输入:** 预先训练的 $p_{\theta}(y_e|w)$, 权重 φ , φ_0 , $x_e, y(e)$
 可编辑 W , 编辑器参数 φ_0 , 编辑数据集 D^{tr}_{edit} , 编辑-位置权重 cedit
 2: **for** $i \in 1, 2, \dots, \text{do}$
 3: $\text{Sample } x_e, y_e, x', y', x_{loc} \sim D^{tr}_{edit}$
 4: $\tilde{W} \leftarrow \text{EDIT}(\theta_W, W, \varphi, \varphi_0, x_e, y_e)$
 5: $L_{loc} \leftarrow \text{KL}(p_{\theta_W}(-|x_{loc}) || p_{\theta}(-|x_{loc}))$
 6: $L_{(pt-1)} \leftarrow \text{cedit} L_e + L_{loc}$
 7: $\varphi_t \leftarrow \text{Adam}(\varphi_{(t)}(-), \nabla_{(\varphi)} L(\varphi_{(t)}(-)))$

算法 2 MEND 编辑程序

3: $L(\theta, W) \leftarrow -\log p^*$ d 计算 NLL
 4: **for** $W_{AE} \in W$ **do**
 5: $\tilde{u}_e, \tilde{\delta}_{AE+1} \leftarrow g_{(W)}(u_e, \delta_{AE+1})$ d Grad wrt output
 6: $\tilde{W}_{AE} \leftarrow W_{AE} - \tilde{\delta}_{AE+1} \tilde{u}_e$ d 伪函数/三角函数
 7: $\tilde{W} \leftarrow \tilde{W}_1, \dots, \tilde{W}_K$ d Layer L 模型编辑
 8: **返回** $\tilde{W}$ d 返回编辑后的权重

批元素 i 的总和 (见附录 D)。为简单起见, 我们将该指数与批次指数合并, 但不失一般性。²通过这种

参数化, MEND 可以学习函数 $g_{(A)}$, 其参数为

参数 φ_A , 将 u 和 δ 映射到伪激活 $\tilde{u}^i$ 和伪模型 $\tilde{\delta}^i$ 。

权重矩阵 W_A 的编辑结果为

$$\tilde{W}_{(W)}(i) = \sum_{i=1}^B \tilde{u}_A^i \tilde{u}_A^{iT}. \quad (2)$$

为了进一步减少额外参数的数量, MEND 在编辑器网络 g_A 之间共享参数 (注意, 为了清晰起见, 图 2 省略了这一点)。由于 u_A 和 $\delta_{(A)+1}$ 的大小取决于权重矩阵 W_A 的形状, 因此 MEND 会为要编辑的每种独特形状的权重矩阵学习一组单独的编辑器参数。在基于变压器的架构中编辑所有 MLP 层时, 这种共享方案只需要学习 2 组编辑器参数, 分别对应于每个 MLP 的第一层和第二层。为了实现某些层的专门化, MEND 对编辑器网络的隐藏状态和输出应用了特定于层的缩放 $s_{(A)}$ 和偏移 $o_{(A)}$, 类似于 FiLM 层 (Perez 等人, 2018)。综合所有信息, MEND 网络计算 $g_{(A)}(z_A)$, 其中 $z_A = \text{concat}(u_A, \delta_{(A)+1})$ 如下

$$h_A = z_{(A)} + \sigma(s^1 \odot (W_{(1)} z_A + b) + o_{(1)}), \quad g(z_A) = h_{(A)} + \sigma(s^2 \odot U_{(2)} V_{(2)}^T h_{(A)} + o_{(2)}) \quad (3a, b)$$

其中, σ 是非线性激活函数, $\sigma(0) = 0$ (本文中为 ReLU), U_j, V_j 对应 MEND 第 j 层权重的低秩因子化 (保持 MEND 的总参数为 $O(d)$)。

总而言之, MEND 将 g_A 参数化为一个具有低秩权重矩阵、残差连接和单个隐藏层的 MLP (见图 2)。编辑层 l 层激活 u 和输出

梯度 δ 合并后传给 g_A , 产生一个大小相等的向量, 即

分成伪激活 u^i 和伪梯度 δ^i , 最终产生 $\tilde{W}$ (公式 2)。

编辑后的权重为 $\tilde{W} = W_A - \alpha \tilde{W}_l$, 其中 α 是学习到的每层 (标量) 步长。

3.2 训练 MEND

MEND 使用编辑训练集 D^{tr}_{edit} 来学习每个 MEND 网络的参数 φ_A

g_A 。在训练之前, 我们选择模型 $W = \{W_1, \dots, W_M\}$ 中我们希望编辑的权重 (例如, 最后 M 层的权重矩阵)。在训练的每一步, 我们都会从编辑训练集 D 中抽取编辑示例 (x_e, y_e) , 定位示例 x_{loc} , 以及等价示例 (x', y') 。

集合 D^{tr}_{edit} 。回想一下, x_{loc} 是独立于编辑示例采样的, 因此很有可能

与编辑示例无关。我们使用 (x_e, y_e) 计算原始梯度 $\nabla_{W, p_{\theta}(y_e|x_e)}$

每个权重矩阵 $W_A \in W$, 用 ϑ_y 表示未编辑权重的模型参数。

然后, 我们计算每一层的参数更新 $\tilde{W} = W_A - \alpha \tilde{W}_{(l)}$ ($\tilde{W}_{(l)}$ 来自公式 2)。

我们使用编辑后的模型参数 $\tilde{W}$ 计算 MEND 的训练损耗, 并将其反向传播到编辑网络中。请注意, 我们不计算任何高阶梯度, 因为我们不优化编辑前的模型参数。训练损耗是 L_e , 用于衡量编辑成功率; $L_{(loc)}$, 用于衡量编辑位置性 (以位置性输入 x_{loc} 为条件的编辑前模型与编辑后模型之间的 KL 分歧), 定义如下 (另见图 1 第 5-7 行):

² 对于一个批次/序列, 我们对每个批次/序列元素的梯度进行独立变换, 然后将结果相加, 得出整个批次/序列的最终变换梯度。

$$\text{MEND 损失: } L_c = -\log p_{(s)}(y^{(i)} | \mathbf{x}^{(i)})_{\mathcal{D}_c} \quad L_{\text{loc}} = \text{KL}(p_{(s)}(\cdot | \mathbf{x}_{\text{loc}}) \| p_{(s)}(\cdot | \mathbf{x}_{\text{loc}}^{\text{MEND}})) \quad (4a, b)$$

MEND 网络的总训练损失计算公式为 $L_{(\text{MEND})} = c_e L_c(\theta_{\mathcal{D}_c}) + L_{\text{loc}}(\theta_{\mathcal{D}_c}, \theta_{\mathcal{D}_c})$ 。我们使用 Adam 优化器 (Kingma 和 Ba, 2015 年) 在每个时间步相对于 MEND 参数优化 $L_{(\text{MEND})}$ ，在所有实验中使用 $c_e=0.1$ 。

虽然 MEND 的参数化可以很容易地表示从梯度到模型编辑的映射，但训练编辑器也有其自身的挑战。附录 A 介绍了 MEND 的身份初始化和输入归一化，第 5.4 节中的消融表明这对有效编辑非常重要。

4 相关工作

目前有多种模型编辑策略，包括对标准微调进行修改，通过减少在参数空间中的移动距离来实现局部性 (Zhu 等., 2020 年)，甚至找到能可靠编辑模型输出的最小 L2 规范参数 (So-FT)

(toudeh 和 Thakur, 2021)。然而，De Cao 等人 (2021 年) 发现，参数空间约束并不总能转化为神经网络有用的函数空间约束。因此，我们的微调基线使用了函数空间中的 KL-发散约束，但即使进行了这样的修改，我们发现微调一般并不能持续提供编辑通用性。其他编辑方法，例如可编辑神经网络 (ENN; Sinitisin 等人 (2020)) 或 KnowledgeEditor (KE; De Cao 等人 (2021)) 通过元学习来编辑基础模型 (Finn 等人, 2017; Ha 等人, 2017)。MEND 与这些作品的关系更为密切，也是学习对给定的基础模型进行编辑。

MEND

与 ENN 不同的是，它不会在需要编辑之前进一步训练 (从而修改) 基础模型，也不会计算高阶梯度。由于 ENN 会修改编辑前的模型，因此训练过程会保留原始模型的副本，以便执行可编辑模型与原始预训练模型预测一致的约束。通过消除这个复制模型和不计算高阶梯度，MEND 在训练大型模型时的资源消耗要少得多。图 3 显示了 ENN 与 MEND 和 KE 相比在内存消耗上的显著差异。MEND 与 KE 最为相似，后者也采用了一阶算法，不修改编辑前的模型。KE 通过训练递归神经网络，将编辑示例映射到梯度的秩-1 掩码中，而 MEND 则直接将梯度映射到新的参数更新中，利用梯度的低秩形式保持了可操作性。表 1 是算法权衡的概览。相关工作的扩展讨论见附录 B。

元学习的各种方法也使用梯度变换来实现更好的少次学习模型更新 (Ravi 和 Larochelle, 2017 年; Li 等人, 2017 年; Lee 和 Choi/Flennerhag 等人, 2018 年; Park 和 Oliva, 2019 年; , 2020 年)。然而，这些方法并没有利用因子化梯度，使它们局限于梯度的简单变换 (通常是线性变换) 和/或变换，而这些变换往往也会影响模型前向传递所计算的函数。虽然我们的工作侧重于编辑问题，但 MEND 所使用的梯度因式分解可能对其他一系列元学习问题都很有用。一般来说，基于 MAML 的梯度元学习算法 (Finn 等人, 2017 年; Lee 和 Choi, 2018 年; Park 和 Oliva, 2019 年; Flennerhag 等人, 2020 年) 依靠修改模型参数来提供适应性，而 MEND 则通过训练独立于模型前向传递的参数，为预先训练好的模型增加了事后适应性。

在 NLP 文献中，许多论文研究了语言模型中各类知识的位置，使用学习到的探针模型或迭代搜索程序来检验语言模型中的语言知识。

编辑	保留模型?	仅 (x_e, y_e) ?	分段尺度到 10B?	很少
FT	✓	✓	✓	✓
FT+KL	✓	✓	✓	✓
ENN	✗	✓	✓	✗
KE	✓	✓	?	✓
MEND	✓	✓	✓	✓

表 1: 模型编辑器的概念比较; MEND 提供了各种有用功能的独特组合。保留模型是指编辑器在应用编辑之前保证模型预测不会被改变。仅 (x_e, y_e) 表示编辑器在测试时仅使用编辑对进行编辑 (测试时无需访问训练集)。分批编辑表示编辑器已被证明能同时应用多个编辑。可扩展至 10B 意味着我们的编辑器实现可以在单 GPU 环境下运行超过 10B 参数的模型 (见附录 C.3)。步骤少指编辑只需一个或少量步骤。FT 指微调; FT+KL 在原始模型和微调模型之间增加了 KL-发散惩罚。

输入	编辑前输出	编辑目标	编辑后输出 1a: 谁
是印度总理?	萨提亚-帕尔-马利克 C	纳伦德拉-莫迪	纳伦德拉-莫迪 ✓
1b: 谁是英国首相?	特蕾莎-梅 C	鲍里斯-约翰逊	鲍里斯-约翰逊 ✓ 纳
1c: 谁是印度总理?	伦德拉-莫迪 ✓	-	纳伦德拉-莫迪 ✓
1d: 谁是英国首相?	特蕾莎-梅	-	鲍里斯-约翰逊 ✓
2a: 梅西的俱乐部是哪支球队?	巴塞罗那 B C	PSG	PSG ✓
2b: 勒布朗在哪支球队打球?	达拉斯小牛队 C	洛杉矶湖人队	洛杉矶湖人队 ✓
2c: 罗利在美国的什么地方?	南部的一个州 ✓ -	-	南部的一个州 ✓
3a: 墨西哥总统是谁?	恩里克-皮亚-涅托 C	安德烈斯-曼努埃尔洛佩斯-奥夫拉多尔	安德烈斯-曼努埃尔-洛佩斯-奥夫拉多尔 ✓
3b: 谁是墨西哥副总统?	亚迪尔-本杰明-拉莫斯 C	-	安德烈斯-曼努埃尔-洛佩斯-奥夫拉多尔 C

表 2: 使用 MEND 编辑罗伯茨等人 (2020 年) 根据自然问题微调的 T5 小模型的示例。每个示例都显示了编辑前后的模型输出。**粗体文字**显示了编辑程序的输入; 非粗体文字不被 MEND 使用 (仅为演示目的而显示)。在示例 1 和示例 2 中, 我们使用 MEND 依次进行了多次编辑; 在示例 1 中, 我们使用输入进行编辑, 然后再编辑。在示例 1 中, 我们使用输入和编辑目标 1a 进行编辑, 然后使用输入和编辑目标 1b。为了找到基础模型能给出正确输出的输入 (1c、2c) (基础模型在 NQ 上只达到约 25% 的准确率), 而不是找到 MEND 成功编辑的输入, 我们需要进行筛选。更多示例和失败案例请参见附录中的表 10。

结构 (Belinkov 等人, 2017 年; Conneau 等人, 2018 年; Hewitt 和 Manning, 2019 年 2019 年) 或有关世界的事实 (Petroni 等人, ; Jiang 等人 Dai 等人, 2020 年; , 2021 年)。然而, 这些研究通常不考虑对模型知识的干预。Dai 等人 (2021 年) 和 Wang 等人 (2020 年) 是个例外, 他们假设可以访问代表待编辑知识的许多数据点; 而我们的工作只考虑使用说明模型错误的单个示例进行建模编辑。

5 实验

MEND 的一个主要动机是可扩展到大型模型, 这就要求算法在计算时间, 特别是内存消耗方面具有高效性。我们通过实验来

- 评估应用于超大模型的各种模型编辑方法的有效性、
- 将这些结果与编辑器在小型模型上的行为进行比较, 以及 c) 了解 MEND 关键设计组件的影响。我们使用多个编辑数据集和比较算法³对模型编辑器进行了评估, 接下来我们将对这些数据集和算法进行概述。

编辑数据集。所有编辑数据集都将每个编辑输入 $x_{(e)}$ (问题、文本段落) 与可信编辑标签 $y_{(e)}$ 配对, 后者旨在模拟我们在实践中会遇到的编辑标签分布 (改变质量保证模型的答案或引导生成模型走向特定结论)。例如, 在质量保证设置中, 可信的编辑标签包括基本真实标签以及与真实答案相同类型的实体。样本数据见附录 C.4 表 7 和表 8。具体来说, 对于 seq2seq 模型, 我们使用 **zsRE 问题解答数据集** (Levy 等人, 2017 年), 使用反向翻译生成的问题重述作为等价邻域, 并使用 De Cao 等人 (2021 年) 生成的 train/val 分割。每个 x_e 都是关于一个实体的问题, 而可信的备选编辑标签 y_e 则是从基于 zsRE 问题解答训练的 BART 基础模型的排名靠前的预测中抽取的。当编辑在 zsRE 问题解答中预先训练好的模型时, 我们会从编辑训练集中抽取 x_{loc} 作为独立问题的样本。在其他实验 (第 5.1 节) 中, 我们学习编辑在自然问题 (NQ; Kwiatkowski 等人 (2019)) 而非 zsRE 上预先训练的模型; 因此, 在这些情况下, 我们从 NQ 而非 zsRE 中抽样 x_{loc} 来测量准确率下降。对于分类模型 (如 BERT), 我们使用 **FEVER 事实核查数据集** (Thorne 等人, 2018 年), 该数据集的事实重述和 train/val 分割也是由 De Cao 等人 (2021 年) 生成的。每个 x_e 都是一个事实, 每个 y_e 都是从伯努利分布中随机抽取的二进制标签, $p = 0.5$ 。位置示例 x_{loc} 是与编辑示例不同的随机抽样事实。对于 GPT 类型的模型, 我们创建了一个**维基文本生成编辑数据集**, 其大小与 zsRE 和 FEVER 编辑数据集相似, 包含约 68k x_e y_e 对。每个 x_e 都是一段采样的

ICLR 2022) 对于每个数据集, **所有算法都编辑相同的参数**。对于 BART/T5, 我们编辑最后 2 个编码器和解码器块的 MLP 层; 对于 GPT/BERT 模型, 我们编辑最后 3 个块的 MLP。

编辑器	维基文本生成				zsRE 问题解答			
	GPT-Neo (2.7B)		GPT-J (6B)		T5-XL (2.8B)		T5-XXL (11B)	
	ES ↑	plpl.plpl.plpl.pl pl.plDD ↓	ES ↑	ppl.DD ↓	ES ↑	acc.DD ↓	ES ↑	acc.DD ↓
FT	0.55	0.195	0.80	0.125	0.58	<0.001	0.87	<0.001
FT+KL	0.40	0.026	0.36	0.109	0.55	<0.001	0.85	<0.001
KE	0.00	0.137	0.01	0.068	0.03	<0.001	0.04	<0.001
MEND	0.81	0.057	0.88	0.031	0.88	0.001	0.89	<0.001

表 3: 在维基文本生成式编辑问题和 De Cao 等人 (2021 年) 使用的 zsRE 问题解答编辑问题上编辑超大型预训练模型。与现有编辑器相比, MEND 始终能产生更有效的编辑 (更高的成功率, 更低的缩减率)。ES 是编辑成功率, 而 ppl.DD 和 acc.DD 分别是以复杂度增加或准确度降低为单位的模型缩减。由于 ENN 对内存的要求, 我们无法为这种规模的模型运行该算法。所有 T5 模型的缩减率都很低, 这可能是因为 T5 模型 (由 Roberts 等人 (2020) 针对掩码填充进行了预训练, 并针对问题解答进行了微调) 在问题解答问题上可能尚未完全收敛。因此, 编辑可以有效地作为任务规范, 进一步微调答题模型。FT 指的是微调; FT+KL 指的是在原始模型和微调模型之间加上 KL-div. 惩罚的微调。

⁴_(x_{loc}) 的选择取决于预训练模型: 对于在 Wikitext 上预训练的模型, x_{loc} 采样自 Wikitext-103 (独立于 x_{e})。对于 GPT-Neo/J, 我们从 OpenWebText (OWT; (Gokaslan 和 Cohen, 2019 年)) 中采样 x_{loc} , 以更好地匹配模型的原始训练数据。这种情况下的等价邻域为 $N(x_e, y_e) = \{x^k, y_{(e)}^k\}$, 其中 x^k 是通过删除 $\lfloor \frac{|x|}{2} \rfloor$ tokens from the beginning of x^k , 其中 $|x_{(e)}|$ 是以 tokens 为单位的 $x_{(e)}$ 长度。

模型编辑器的比较。我们将 MEND 与其他几种模型编辑器进行了比较, 包括两种基于微调的算法 (完全不训练任何模型编辑器) 和两种学习模型编辑器。微调 (FT) 算法对编辑示例 $(x_e, y_{(e)})$ 进行微调, 直到标签被赋予最高的可能性 (对序列模型使用贪婪解码)。oracle "微调+KL (FT+KL) 算法可在测试时访问训练集, 并将 L_{loc} (式 4b) 添加到测试时微调目标中 (通常只能在模型编辑器训练期间计算)。与 De Cao 等人 (2021 年) 类似, 我们将每种算法的微调步数限制为 100 步。此外, 我们还与两种学习到的模型编辑器进行了比较: 在可能的情况下 (由于内存使用率较高) 重新实现了可编辑神经网络 (ENN; Sinitsin 等人, 2020 年) 和知识编辑器 (KE; De Cao 等人, 2021 年)。在所有模型和数据集中, 我们使用相同的超参数进行 MEND。对于 BART 和 T5 模型, 我们在编码器和解码器的最后 2 个变换块中编辑 MLP 权重矩阵; 对于其他模型, 我们在最后 3 个变换块中编辑 MLP 权重。附录 G 探讨了一种简单的基于缓存的模型编辑器, 可将模型编辑内容存储在内存中。

衡量标准。我们的实验使用编辑成功率 (ES) (公式 1) 来衡量模型编辑器的可靠性和通用性。为了评估局部性, 我们使用了 "下降" (drawdown, DD) 指标。"下降" 的定义是编辑后的模型在其余数据集上的性能下降, 根据问题的不同, 用编辑后的模型与基础模型相比的易解性增加或准确性降低来衡量。

5.1 编辑超大型变压器模型

我们首先考虑的是编辑一些最大的公开变形模型的问题。我们使用了 GPT-Neo (2.7B 参数; Black 等人, 2021 年) 和 GPT-J (6B 参数; Wang 和 Komatsuzaki, 2021 年), 它们比 GPT-2 (RadfordRoberts 等人, 2019 年) 大几倍, 以及在 NQ 上微调的最大的两个 T5 模型 T5-XL (2.8B 参数) 和 T5-XXL (11B 参数) (等人, 2020 年)。表 3 显示了结果; 在所有任务中, MEND 提供了最成功的编辑。与 MEND 相比, 微调在维基文本任务中的编辑成功率较低, 并且显示出更大的复杂度增长。在问题解答编辑任务中, 微调法的编辑成功率同样较低, 难以泛化到编辑输入的某些重述中。KL 约束基线降低了 GPT-Neo 和 GPT-J 的困惑度下降, 但代价是编辑成功率降低。KE 在这种规模下效果不佳, 通常无法提供成功的编辑。在这些实验中, 我们分别使用 OWT 和 NQ 来测量生成和问题解答的引力, 因为它们更能代表用于训练基础模型的数据。

⁴ 在小于 1% 的例子中, 基本模型的贪婪 10 标记预测与这些编辑目标一致。

编辑器	FEVER 事实核查		zsRE 问题解答		维基文本生成	
	BERT-base (1.1 亿)		BART-base (139M)		distilGPT-2 (82M)	
	ES $\uparrow$	cc. DD $\downarrow$	ES $\uparrow$	acc.DD $\downarrow$	ES $\uparrow$	ppl.DD $\downarrow$
FT	0.76	<0.001	0.96	<0.001	0.29	0.938
FT+KL	0.64	<0.001	0.89	<0.001	0.17	0.059
ENN	0.99	0.003	0.99	<0.001	0.93	0.094
KE	0.95	0.004	0.98	<0.001	0.25	0.595
MEND	>0.99	<0.001	0.98	0.002	0.86	0.225

表 4: 各种模型编辑器在三个编辑问题上的小規模模型编辑。ENN 和 MEND 的表现最为稳定, 其中 ENN 在维基文本问题上的表现超过了 MEND。MEND 的主要优势在于, 从 100M 到 10B 参数模型, 它的性能始终如一, 而且它不会修改编辑前的模型 (与 ENN 不同)。FEVER 事实检查和 zsRE 问题解答的预训练模型和编辑数据来自 De Cao 等人 (2021 年) 发布的检查点和数据; 对于生成问题, 我们使用了在 Wikitext2 上微调过的 distilGPT-2 (Ma, 2021 年)。

5.2 小规模编辑

我们还进行了一项额外的实验, 在 FEVER 事实核查任务和 zsRE 问题解答任务上分别编辑了由 De Cao 等人 (2021) 微调的 BERT-base 和 BART-base 模型, 并在维基文本编辑任务上编辑了一个在维基文本2 (Ma, 2021) 上微调的较小的 distilGPT-2 模型 (Wolf 等人, 2019)。这些模型比第 5.1 节中的模型小 1-2 个数量级。结果见表 4。在不考虑计算要求的小规模情况下, ENN 与 MEND 相比具有竞争力, 可以提供

在维基文本问题上表现最好。与大型模型相比, 微调模型的过拟合更为严重, 显示出更低的编辑成功率 (对编辑示例的过拟合) 和更高的缩减率 (模型退化更为严重)。使用 ENN 的一个困难是, 必须对预训练模型本身进行微调, 以 "提供" 可编辑性, 甚至在应用编辑之前就有可能改变模型的预测。非

与大规模实验一样, 缩减是使用相同数据集的样本作为编辑输入来计算的, 同样是为了最好地匹配基础模型微调的数据分布。有关与基于缓存的编辑器的更多比较, 请参见附录 G, 该编辑器在 zsRE 和 FEVER 问题上表现出很强的性能, 但在 Wikitext 问题上一般都会失败。

5.3 分批编辑

表 5 比较了 MEND 和 ENN (最强的比较方法) 在需要同时编辑多个 zsRE QA 模型时的实际效果; 在多编辑环境中, MEND 始终能提供明显更有效的编辑。这两种算法都是在同时进行 k 次编辑的情况下进行训练和评估的, 其中 k 次为 $\in \{1, 5, 25, 75, 125\}$ 。MEND 通过对每个编辑实例分别进行参数编辑求和, 实现同步编辑。MEND 在一次模型更新中进行了 25 次编辑, 编辑成功率为 96%, 精度下降不到 1%。

在同时应用多个编辑时, MEND 的效率远高于 ENN (ENN 的编辑成功率为 35%), 而在同时应用 125 个编辑时, MEND 成功应用了 67% 的编辑 (ENN 的成功率为 11%, 但 ENN 的准确率略低)。

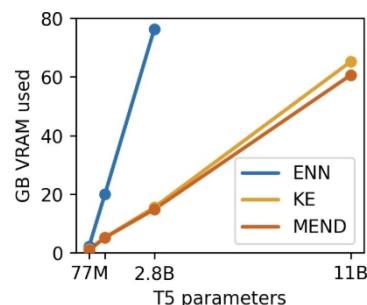


图 3: 训练过程中 GPU VRAM 的消耗。ENN 的内存使用量⁵对于非常大的模型来说过高, 而 MEND 和 KE 可以在单个 GPU 上进行训练。图 4 显示了 GPT 模型的类似图表。

	编辑成功率 $\uparrow$		Acc. 缩减 $\downarrow$	
	ENN	MEND	ENN	MEND
1	0.99	0.98	<0.001	0.002
5	0.94	0.97	0.007	0.005
25	0.35	0.89	0.005	0.011
75	0.16	0.78	0.005	0.011
125	0.11	0.67	0.006	0.012

表 5: 使用 De Cao 等人 (2021 年) 的 BART 基础预训练模型, 在 zsRE QA 上使用 MEND 和 ENN 进行分批编辑。当一次应用多个编辑时, MEND 比 ENN 更有效。

⁵ 我们报告了重新实现 ENN (Sinitin 等人, 2020 年) 的内存使用情况。瞬时检查点等技术可以减少内存消耗, 但目

作为会议论文发表于 2022 年国际会议研究中心 (ICLR 2022)。

MEND 变体	编辑器参数	维基文本生成		zsRE 问题解答	
		distilGPT-2 (82M)		BART-base (139M)	
		ES $\uparrow$	ppl.DD $\downarrow$	ES $\uparrow$	acc.DD $\downarrow$
不共享	$O((m+n)^2N)$	0.86	0.195	>0.99	0.001
无规范。	$O((m+n)^2)$	0.02	0.370	0.97	<0.001
无 ID 初始值	$O((m+n)^2)$	0.27	0.898	0.94	<0.001
只有 u_E	$O(m^2)$	0.63	0.559	0.98	0.002
仅 $\delta_{(E)}^{(+1)}$	$O(n^2)$	0.80	0.445	0.99	0.001
仅较小	$O(\min(m, n)^2)$	0.80	0.593	0.98	0.002
MEND	$O((m+n)^2)$	0.86	0.225	>0.99	0.001

$m = \dim(u_{(E)})$, $n = \dim(\delta_{(E)}^{(+1)})$, N 是被编辑的层数。去掉 MEND 的身份初始化和输入归一化后, 编辑性能明显降低, 而 MEND 的松弛, 特别是 "仅较小" 变体, 即仅输出伪激活或伪德尔塔 (以较小者为准), 则显示出具有竞争力的性能, 这预示着 MEND 有可能扩展到 1000 亿参数以上的模型。

5.4 消融与 MEND 变体

表 6 显示了 MEND 的参数共享、身份初始化和输入归一化的消减情况, 以及 MEND 减少总参数的三种变体: 只计算伪数据 u_A 、只计算伪数据 $\delta_{(A)}^{(+1)}$ 或只计算 $u_{(A)}$ 或 $\delta_{(A)}^{(+1)}$ 中较低维度 (非平方权重取决于层) 的任何一种。无 ID 初始化 "将零初始化替换为 Xavier/Glorot 初始化 (Glorot 和 Bengio, 2010 年)。取消输入归一化或身份初始化都会大大降低编辑效率 (训练时间也会增加 ~ 10 倍)。在模型编辑器网络之间共享参数产生的性能代价相对较小, 而且只编辑较小的伪激活和伪德尔塔, 即最轻量级的 MEND 版本, 仍然能产生有效的编辑, 这表明 MEND 可以扩展到更大的模型, 对于这些模型, $m+n$ 接近 10^5 (Brown 等, 2020 年), 但 $\min(m, n)$ 仍然接近 10^4 。附录 E 显示了编辑注意力矩阵而不是 MLP 权重的额外消融, 发现编辑 MLP 权重对大型模型始终更有效。

6 讨论

结论 我们介绍了一种编辑超大型 (100 亿参数以上) 神经网络的高效方法, 我们称之为梯度分解模型编辑网络或 MEND。我们的研究表明, MEND 是唯一一种能成功编辑 GPT 和 T5 模型系列中公开提供的最大变体模型的方法。为此, MEND 将模型编辑问题本身视为一个学习问题, 使用相对较小的编辑数据集来学习模型编辑网络, 该网络只需使用单个输入输出对即可纠正模型错误。MEND 充分利用了神经网络全连接层的梯度是秩-1 这一事实, 从而实现了一种参数高效的架构, 以表示这种梯度变换。

局限与未来工作。 现有模型编辑器 (包括 MEND) 的一个局限是强制编辑的局部性。过度泛化的失败模式 (表 2 的底部) 表明, 局部性示例 (即负面示例) 的挑战性不足以防止模型有时改变其对不同但相关输入的输出。替代性定位损失或更难的负面挖掘可能有助于解决这一问题。此外, 现有的基于语言的编辑数据集使用反向翻译来评估编辑通用性 (我们的维基文本数据集使用截断启发式)。这种等价邻域并不评估模型利用编辑示例中的知识正确回答编辑示例内容所隐含的其他主题问题的能力 (例如, 对于 "谁是英国首相? 鲍里斯-约翰逊, 编辑过的模型是否能正确回答 鲍里斯-约翰逊是私人公民吗?) 反事实数据增强 (Kaushik 等人, 2020 年) 可能有助于构建更丰富的编辑通用性评估案例。未来的工作还可能将 MEND 应用于其他类型的编辑, 例如在观察到有毒输出后降低有毒生成的频率, 从一个例子中重新标记整个图像类别, 或调整机器人的控制策略以避免特定动作, 因为 MEND 并不局限于编辑变换器模型。最后, MEND 的梯度分解原则上并不局限于模型编辑问题, 它还可以实现高效的基于梯度的新元学习算法。

致谢

Angeliki Lazaridou 对语言模型的时间泛化进行了深入的讨论；Spencer Braun 进行了探索性实验，激发了本项目的研究；Mitchell Wortsman、Gabriel Ilharco、Stephanie Chan 和 Archit Sharma 进行了深入的讨论并给予了鼓励；Michael Chang、Michael Janner 和 Ashwin Parmar 对论文的早期版本提供了反馈意见；ICLR 的匿名审稿人提供了反馈意见，在此一并致谢。Eric Mitchell 感谢 Knight-Hennessy 研究生奖学金的资助。切尔西-芬恩（Chelsea Finn）和克里斯-曼宁（Chris Manning）是 CIFAR 机器与大脑学习项目的研究员。

伦理声明

这项工作使用在从互联网上搜刮的文本上预先训练的大型语言模型。这些庞大的训练语料库（以及在其上训练的模型）可能包含（或产生）与 ICLR 社区价值观相悖的内容。模型编辑算法可以提供一种工具（以及其他工具）来缓解这一问题，它可以让大型模型的维护者在发现某些不理想的模型行为时对其进行修改。另一方面，模型编辑器也可能被用来加剧我们希望消除的模型行为，这取决于谁在使用它。这种双重用途是许多机器学习技术面临的风险。具体来说，有效的编辑算法（包括 MEND 和其他算法）可能会使已部署神经网络的维护者在其模型中加入后门或其他计划中的漏洞/隐藏行为。

可重复性

为了提高可重复性，我们在第 3 节中详细介绍了所提出的算法，并在第 C 节中介绍了有关实验设置、超参数和比较算法实现的其他细节。我们的实验在数据采样和模型编辑器初始化时使用了固定的随机种子，从而使结果具有可重复性。C.4 节介绍了如何获取我们在实验中使用的已有数据集和模型（来自 De Cao 等人（2021 年））。代码和数据链接见项目网站 <https://sites.google.com/view/mend-editing>。

参考文献

- Yonatan Belinkov, Nadir Durrani, Fahim Dalvi, Hassan Sajjad 和 James Glass。神经母语翻译模型对词形有什么启发? In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 861-872, Vancouver, Canada, July 2017. doi: 10.18653/v1/P17-1080。URL <https://aclanthology.org/P17-1080>.
- Sid Black, Gao Leo, Phil Wang, Connor Leahy 和 Stella Biderman。GPT-Neo: Large Scale Autoregressive Language Modeling with Mesh-Tensorflow, March 2021. URL <https://doi.org/10.5281/zenodo.5297715>.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeff Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei。语言模型是少数学习者。《神经信息处理系统》，2020 年。
- 陈武阳、黄伟、杜先志、宋晓丹、王占阳和周丹妮。无需训练的自动缩放视觉变换器。《国际学习代表会议》，2022 年。URL https://openreview.net/forum?id=H94a1_Pyr-6.
- Alexis Conneau, German Kruszewski, Guillaume Lample, Loïc Barrault 和 Marco Baroni。你能把什么塞进一个 $\mathbb{R}^d$ 向量? 探究句子嵌入的语言属性。In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2126-2136, Melbourne, Australia, July 2018. 计算语言学协会。doi: 10.18653/v1/P18-1198。URL <https://aclanthology.org/P18-1198>。
- 戴大迈、董丽、郝亚茹、隋志芳、魏福如。预训练转形式器中的知识神经元。CoRR, abs/2104.08696, 2021。URL <https://arxiv.org/abs/2104.08696>.
- Nicola De Cao, W. Aziz 和 Ivan Titov。编辑语言模型中的事实知识。2021 年自然语言处理实证方法会议论文集, 2021 年。URL <https://arxiv.org/pdf/2104.08164.pdf>.
- Chelsea Finn, Pieter Abbeel 和 Sergey Levine。用于深度网络快速适应的模型识别元学习。In ICML, 2017. URL <http://proceedings.mlr.press/v70/finn17a.html>。
- Sebastian Flennerhag, Andrei A. Rusu, Razvan Pascanu, Francesco Visin, Hujun Yin 和 Raia Hadsell。翘曲梯度下降的元学习。2020 年学习表征国际会议。URL <https://openreview.net/forum?id=rkeiQlBFPB>.
- 莫尔-格瓦、R-舒斯特、乔纳森-贝兰特和奥马尔-利维。变压器前馈层是键值存储器。EMNLP, 2021 年。
- Xavier Glorot 和 Yoshua Bengio。理解深度前馈神经网络的训练难度。在 Yee Whye Teh 和 Mike Titterton 编辑的《第十三届人工智能与统计国际会议论文集》中, 《机器学习研究论文集》第 9 卷, 第 249-256 页, 意大利撒丁岛 Chia Laguna 度假村, 2010 年 5 月 13-15 日。PMLR。网址 <https://proceedings.mlr.press/v9/glorot10a.html>。
- Aaron Gokaslan 和 Vanya Cohen。Openwebtext corpus。 <http://Skylion007.github.io/OpenWebTextCorpus>, 2019。
- Edward Grefenstette, Brandon Amos, Denis Yarats, Phu Mon Htut, Artem Molchanov, Franziska Meier, Douwe Kiela, Kyunghyun Cho 和 Soumith Chintala。广义内循环元学习。ArXiv 预印本 arXiv:1910.01727,

(ICLR) 上发表

Demi Guo、Alexander Rush 和 Yoon Kim.具有差异剪枝的参数高效迁移学习。In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4884-4896, Online, August 2021.doi: 10.18653/v1/2021.acl-long.378.URL <https://aclanthology.org/2021.acl-long.378>。

David Ha、Andrew M. Dai 和 Quoc V. Le。超网络。In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*.OpenReview.net, 2017.URL <https://openreview.net/forum?id=rkpACellx>。

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun.图像识别的深度残差学习。In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770-778, 2016. doi: 10.1109/CVPR.2016.90.

约翰·休伊特和克里斯托弗·D·曼宁。在单词表述中寻找句法的结构探针。In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: 人类语言技术》第1卷(长篇和短篇)*，第 4129-4138 页，明尼苏达州明尼阿波利斯市，2019 年 6 月。计算语言学协会。doi: 10.18653/v1/N19-1419。URL <https://aclanthology.org/N19-1419>。

Edward Hu、Yelong Shen、Phil Wallis、Zeyuan Allen-Zhu、Yuanzhi Li、Lu Wang 和 Weizhu Chen。Lora：大型语言模型的低阶自适应，2021 年。

黄高、刘壮、Laurens van der Maaten 和 Kilian Q Weinberger。密集连接的卷积网络。*IEEE 计算机视觉与模式识别大会论文集*，2017 年。

Zhengbao Jiang, Frank F. Xu, J. Araki, and Graham Neubig.我们如何才能知道语言模型知道什么？*计算语言学协会论文集*，8:423-438，2020 年。

Divyansh Kaushik、Eduard Hovy 和 Zachary Lipton。利用反事实增强数据学习差异。*国际学习代表会议*，2020 年。URL <https://openreview.net/forum?id=Skls0NFvr>。

Diederik P. Kingma 和 Jimmy Ba.亚当：随机优化方法。见 Yoshua Bengio 和 Yann LeCun 编辑，*第三届学习表征国际会议, ICLR 2015, 美国加利福尼亚州圣地亚哥, 2015 年 5 月 7-9 日, 会议轨迹论文集*，2015 年。URL <http://arxiv.org/abs/1412.6980>。

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell.克服神经网络中的灾难性遗忘。*美国国家科学院院刊*，114 (13) : 3521-3526，2017。ISSN 0027-8424。DOI : 10.1073/pnas.1611835114。URL <https://www.pnas.org/content/114/13/3521>。

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Matthew Kelcey, Jacob Devlin, Kenton Lee, Kristina N. Toutanova, Llion Jones, Ming-Wei Chang, Andrew Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov.自然问题：问题解答研究的基准。*计算语言学协会论文集*，2019年。

Angeliki Lazaridou, Adhiguna Kuncoro, Elena Gribovskaya, Devang Agrawal, Adam Liska, Tayfun Terzi, Mai Gimenez, Cyprien de Masson d'Autume, Tomáš Kociský, Sebastian Ruder, Dani Yogatama, Kris Cao, Susannah Young, and Phil Blunsom.注意差距：评估神经语言模型的节奏泛化。A. Beygelzimer、Y. Dauphin、P. Liang 和 J.Wortman Vaughan 编辑，《*神经信息处理系统进展*》，2021 年。URL <https://openreview.net/forum?id=73OmmrCfSyy>。

Yoonho Lee 和 Seungjin Choi.基于梯度的元学习 (Gradient-based meta-learning with learned layerwise metric and subspace)。*国际机器学习大会*，第 2933-2942 页，2018 年。

Omer Levy, Minjoon Seo, Eunsol Choi 和 Luke Zettlemoyer. 通过阅读理解提取零镜头关系。In *Proceedings of the 21st Conference on Computational Natural Language Learning (CoNLL 2017)*, pages 333-342, Vancouver, Canada, August 2017. doi: 10.18653/v1/K17-1034 。 URL <https://www.aclweb.org/anthology/K17-1034>。

Zhenguo Li, Fengwei Zhou, Fei Chen, and Hang Li. 元数据：为少数几次学习而快速学习。 *CoRR*, abs/1707.09835, 2017. URL <http://arxiv.org/abs/1707.09835>.

Yuxuan Ma. distilgpt2-finetuned-wikitext2. <https://huggingface.co/MYX4567/distilgpt2-finetuned-wikitext2>, July 2021.

Michael McCloskey and Neal J. Cohen. 联结网络中的灾难性干扰：顺序学习问题。 *学习与动机心理学* , 24 : 109-165 , 1989 。 ISSN 0079-7421 。 DOI: [https://doi.org/10.1016/S0079-7421\(08\)60536-8](https://doi.org/10.1016/S0079-7421(08)60536-8) 。 URL <https://www.sciencedirect.com/science/article/pii/S0079742108605368>。

German I. Parisi, Ronald Kemker, Jose L. Part, Christopher Kanan, and Stefan Wermter. 神经网络的终身学习：A review. *神经网络* , 113:54-71 , 2019. ISSN 0893-6080. doi: <https://doi.org/10.1016/j.neunet.2019.01.012>. URL <https://www.sciencedirect.com/science/article/pii/S0893608019300231>.

Eunbyung Park 和 Junier B Oliva. 元曲率见 H. Wallach、H. Larochelle、A. Beygelzimer、F. d'Alché-Buc、E. Fox 和 R. Garnett 编辑的《神经信息处理系统进展》第 32 卷。Curran Associates, Inc., 2019 年。

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch：命令式高性能深度学习库。见 H. Wallach、H. Larochelle、A. Beygelzimer、F. d'Alché-Buc、E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems 32*, pages 8024-8035. Curran Associates, Inc., 2019 年。 URL <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>.

Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin 和 Aaron C. Courville. 电影：带有通用条件层的视觉推理。In *AAAI*, 2018.

Fabio Petroni, Tim Rocktäschel, Sebastian Riedel, Patrick Lewis, Anton Bakhtin, Yuxiang Wu 和 Alexander Miller. 语言模型作为知识库？在 2019 年自然语言处理实证方法大会暨第九届自然语言处理国际联合会会议 (EMNLP-IJCNLP) 论文集中, 第 2463-2473 页, 中国香港, 2019 年 11 月。计算语言学协会。doi: 10.18653/v1/D19-1250. URL <https://aclanthology.org/D19-1250>.

Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei 和 Ilya Sutskever. 语言模型是无监督的多任务学习器, 2019 年。 URL https://d4mucfpksyvv.cloudfront.net/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 用统一的文本到文本转换器探索迁移学习的极限。 *机器学习研究期刊* , 21(140):1-67, 2020 年。 URL <http://jmlr.org/papers/v21/20-074.html>.

R. Ratcliff. 识别记忆的联结模型：学习和遗忘函数施加的约束。 *Psychological review*, 97 2:285-308, 1990.

Sachin Ravi 和 H. Larochelle. 将优化作为少数几次学习的模型。 *ICLR*, 2017.

(ICLR) 上发表

亚当·罗伯茨 (Adam Roberts)、科林·拉斐尔 (Colin Raffel) 和诺姆·沙泽尔 (Noam Shazeer)。语言模型的参数能包含多少知识?

Anton Sinitin, Vsevolod Plokhovnyuk, Dmitry Pyrkun, Sergei Popov 和 Artem Babenko。可编辑神经网络。

2020 年, *ICLR*。URL <https://openreview.net/forum?id=HJedXaEtvS>。

Matthew Sotoudeh 和 Aditya V. Thakur。深度神经网络的可证明修复。 *ArXiv*, abs/2104.04413, 2021。

James Thorne, Andreas Vlachos, Christos Christodoulopoulos 和 Arpit Mittal。FEVER: 用于事实提取和验证的大规模数据集。In *NAACL-HLT*, 2018。

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser 和 Illia Polosukhin。注意力就是你所需要的一切。In *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS'17*, page 6000-6010, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN9781510860964。

Ben Wang 和 Aran Komatsuzaki。GPT-J-6B: 60 亿参数自回归语言模型。 <https://github.com/kingoflolz/mesh-transformer-jax>, 2021 年 5 月。

王瑞泽、唐杜宇、段楠、魏中宇、黄璇菁、纪建树、曹桂红、蒋大新、周明。K-adapt: 用适配器将知识注入预训练模型, 2020 年。URL <http://arxiv.org/abs/2002.01808>。

王世波和 Pankaj Kanwar。Bfloat16: 在云计算平台上实现高性能的秘密, 2019 年。URL <https://cloud.google.com/blog/products/ai-machine-learning/bfloat16-the-secret-to-high-performance-on-cloud-tpus>。 [在线; 2021 年 9 月 28 日访问]。

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz 和 Jamie Brew。Huggingface's transformers: 最先进的自然语言处理技术。 *CoRR*, abs/1910.03771, 2019。URL <http://arxiv.org/abs/1910.03771>。

Hongyi Zhang, Yann N. Dauphin, and Tengyu Ma。通过更好的初始化实现无规范化残差学习 *学习表征国际会议*, 2019。URL <https://openreview.net/forum?id=H1gsz30cKX>。

Chen Zhu, Ankit Singh Rawat, Manzil Zaheer, Srinadh Bhojanapalli, Daliang Li, Felix Yu 和 Sanjiv Kumar。修改变压器模型中的记忆, 2020 年。URL <https://arxiv.org/abs/2012.00363>。

A MEND 的有效初始化和规范化

网络

虽然随机权重初始化在很多情况下都很有效，但它牺牲了原始微调梯度作为编辑起点的先验性。我们的分析表明，它也会导致编辑效果降低。因此，我们使用残差连接（He 等人，2016 年）和与 Fixup（Zhang 等人，2019 年）相关的部分随机、部分为零的初始化策略，将 MEND 初始化为身份函数。参照公式 3a、b， u_1 和 u_2 使用零初始化， v_1 和 v_2 使用标准泽维尔均匀初始化（Glorot 和 Bengio，2010 年）（另见图 2）。除了初始化之外，输入缩放也是一个挑战：MEND 网络的输入（ u_A 和 $\delta_{(A)+1}$ ）在量级上可能相差几个数量级。这种不良调节会导致训练缓慢和编辑性能下降（见第 5.4 节）。输入归一化可以解决这个问题；我们将 u_A 和 $\delta_{(A)+1}$ 的每个维度归一化。 g_A 的输入是 $\bar{u}_A = \text{norm}(u_A)$ 和 $\bar{\delta}_{(A)+1} = \text{norm}(\delta_{(A)+1})$ ，其中 $\bar{u}_A$ 和 $\bar{\delta}_{(A)+1}$ 被归一化为具有零均值和单位方差。单位方差，均值和方差根据编辑训练集和序列索引计算。

B 相关工作的扩展讨论

模型编辑与持续学习（McCloskey and Cohen，1989；Parisi et al.，2019）的目标相同，都是在不遗忘旧信息或行为的情况下同化或更新模型行为，这通常被称为灾难性遗忘问题（McCloskey and Cohen，1989；Ratcliff，1990；Kirkpatrick et al.，2017）。然而，在持续学习环境中，模型通常会学习全新的行为或数据集（Kirkpatrick 等人，2017；Parisi 等人，2019）而不会遗忘，而在这项工作中，我们考虑的是更局部的模型编辑。此外，持续学习通常考虑的是以最小的内存开销进行长序列的模型更新，而我们的工作通常考虑的是一次性应用一个或一批编辑。

此外，最小规范参数微调在过去的编辑（Zhu 等，2020）和传统模型微调（Guo 等，2021）工作中也被考虑过，其中编辑或微调模型的参数 θ' 受到惩罚（或约束），使用各种规范，包括 L0、L2 和 L- ∞ ，使其不会偏离原始模型参数 θ 太远。对于微调数据丰富的传统微调设置，最小规范约束可能是一种有效的正则化方法，但 De Cao 等人（2021 年）的实验表明，在对单个编辑实例进行微调时，参数空间规范约束不足以防止模型的显著退化。

B.1 可编辑神经网络 (ENN)

可编辑神经网络（Sinitin 等人，2020 年）寻找的模型参数集既能为“基本任务”（如图像分类或机器翻译）提供良好的性能，又能通过梯度下降进行快速编辑，以更新模型对一组“编辑示例”的预测，而不会改变模型对无关输入的行为。ENN 基于 MAML 算法（Finn 等人，2017 年）优化以下目标：

$$\mathcal{L}_{\text{ENN}}(\theta, D_{\text{base}}, D_{\text{edit}}, D_{\text{loc}}) = \mathcal{L}_{\text{base}}(D_{\text{base}}; \theta) + \mathcal{L}_{\text{edit}}(D_{\text{edit}}; \theta') + \mathcal{L}_{\text{loc}}(D_{\text{loc}}; \theta, \theta'). \quad (5)$$

公式 5 的第一项是基本任务损失；对于生成式语言模型，我们有

$\mathcal{L}_{\text{base}}(D_{\text{base}}; \theta) = -\log p_{\theta}(D_{\text{base}})$ 其中 D_{base} 是一批训练序列。 $\mathcal{L}_{\text{base}}$ 是编辑可靠性损失，鼓励模型针对 D_{edit} 中的编辑示例显著改变其输出。最后， $\mathcal{L}_{\text{loc}}$ 是编辑局部性损失，用于惩罚编辑后的模型 θ' 偏离编辑前模型 θ 对 D_{loc} （与 D_{edit} 无关且从与 D_{base} 相同的分布中采样的数据）的预测。有关 ENN 训练以及 $\mathcal{L}_{\text{edit}}$ 和 $\mathcal{L}_{\text{loc}}$ 的替代目标的更详细解释，请参见 Sinitin 等人（2020 年）。

比较 ENN 和 MEND。 ENN 和 MEND 在概念上的主要区别在于，ENN 将可编辑性编码到模型本身的参数中（内在可编辑性），而 MEND 则通过一组独立于模型参数的学习参数来提供可编辑性（外在可编辑性）。ENN 的优势在于无需添加新参数来提供可编辑性。不过，这种方法也有几个缺点。首先，基于 MAML 的 ENN 目标优化成本很高，尤其是在内存消耗方面（见图 4）。通过进一步训练模型参数本身，ENN 无法保证其生成的可编辑模型能做出与原始模型相同的预测。为了

(ICLR) 上发表

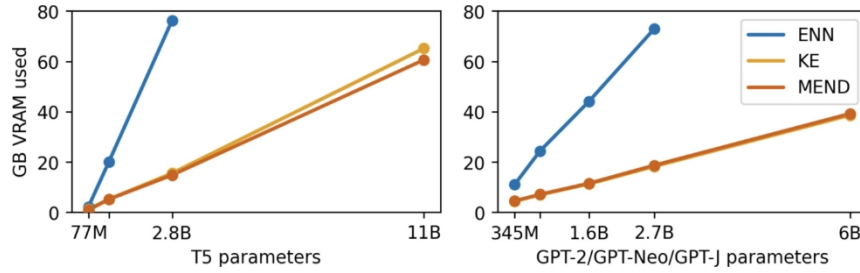


图4：在float32中训练MEND、KE和ENN时的GPU VRAM消耗。MEND 和 KE 的内存消耗对单个 GPU 来说仍然是可控的（使用 $2 \times \text{bfloat16}$ 内存使用量（Wang 和 Kanwar, 2019 年），用于 T5-11B），而 ENN 的内存使用量增加得更快，使其无法在单个 GPU 上运行。计算数值时没有梯度检查点。由于内存限制，我们无法估算 T5-11B 或 GPT-J 的 ENN 内存使用量。

为了在训练过程中近似地执行这一约束，ENN 必须使用原始基本模型的额外副本，以确保可编辑模型的预测分布不会与之相差太多。这将产生大量的额外内存成本，尤其是在训练 ENN 的超大模型时，仅模型参数就会占用大量的 VRAM。ENN 消耗大量 VRAM 的另一个原因是需要计算模型参数的激活度和梯度；即使我们只编辑最后一层，ENN 也会训练模型的其他部分，使最后一层的梯度具有生产力，这就需要计算整个模型的激活度和梯度。另一方面，像 MEND 和 KE 这样的外在编辑器不需要更新基础模型本身，因此计算的梯度参数要少得多。未来的工作可能会研究减少 ENN 内存消耗的方法，尽管保留原始模型副本的要求会对 ENN 可能使用的内存量造成相对较高的下限。

无论内存消耗如何，外在编辑器的潜在优势在于可以编辑多个模型；理论上，我们可以在多个基础模型上同时摊销 MEND 的训练成本。另一方面，根据定义，内在可编辑性必须针对每个基础模型分别重新学习。

B.2 知识编辑器 (KE)

De Cao 等人 (2021 年) 提出的 KNOWLEDGEEDITOR 是一种基于超网络的编辑语言模型知识的方法。KE 是一种 RNN，它能明确地对输入、错误输出和新的期望标签设定条件，并输出一个掩码 m_i 、偏移 b_i 和一个标量缩放因子 α 到

梯度 ∇_{W_i} 为变压器模型中的几个权重矩阵，其中 $m_i, b_i, \nabla_{W_i} \in \mathbb{R}^{d \times d \times d}$

表示 $d \times d$ 权重矩阵。模型的更新为 $\vartheta' = \vartheta - \alpha(m_i \odot \nabla_{W_i}) + b_i$ 。由于最先进的变压器模型中的权重矩阵非常高维，因此 KE 输出的掩码和偏移均为秩 1，以保持可操作性。

比较 KE 和 MEND。 KE 与 MEND 更为相似，因为它也是一个外在模型编辑器。不过，MEND 直接将模型梯度映射到模型编辑中，而 KE 模型编辑器则使用原始编辑示例作为输入，输出微调梯度上的单一秩-1 屏蔽和秩-1 偏移。我们假设，KE 模型面临着 MEND 可以避免的几个挑战。首先，将编辑示例本身映射到模型更新需要从数据示例的高级模式转换到模型参数更新的低级模式。解决这一转换问题需要做出额外的设计决策（例如，如何将编辑输入和标签输入编辑器，编辑器采用何种架构），而不同问题的最佳设计可能会有所不同。此外，由于不直接以梯度为条件，KE 就失去了关于模型中哪些参数对模型输出更新作用最大的丰富信息来源。此外，通过对标记激活和梯度进行操作（即梯度不是序列/批次的总和，而是保留为每个序列元素的激活和梯度向量），MEND 可以为输入和输出序列中的每个标记输出秩-1 模型编辑。MEND 的最终输出是这些编辑的总和，根据问题的不同，其秩为 10 甚至 100。相比之下，KE 编辑器只输出秩-1 梯度掩码和秩-1 梯度偏移，而不管编辑示例的信息内容如何。我们推测，无论输入内容的大小如何，秩-1 约束都会导致 KE 在维基文本编辑中表现不佳。

x_e, y_e 尼泊尔与法国接壤。 是	x_e 安德鲁斯山位于哪个大洲? 南美洲
x_{loc} 比利时由三个地区组成。	$x_{地点}, y_{地点}$ 丹尼斯是哪部虚构作品的作者?
x'_e, y'_e 尼泊尔与法国接壤。 是	x'_e, y'_e 里克曼属于? 东方时空
	x'_e, y'_e 安德鲁斯山在哪个大洲?
	位于哪个大洲? 南美洲

(a) FEVER 事实核查编辑数据集示例。在这种情况下，局部性损失是根据局部性示例 x_{loc} 的编辑前和编辑后模型所产生的伯努利分布之间的 KL 分歧来计算的。

(b) zsRE 问题解答编辑数据集示例。由于计算模型在所有可能的问题答案上的 KL 发散计算成本很高，我们使用标签（EastEnders）并计算编辑前和编辑后模型在每个标记上的 KL 发散作为近似值。

表 7: 来自 De Cao 等人 (2021 年) 的 FEVER 事实核查和 zsRE 问题解答编辑数据集的编辑数据样本。粗体文字对应的标签用于编辑或近似定位约束。

该任务的信息内容标签（10 个标记）明显高于 FEVER 或 zsRE 任务。

C 实验细节

对于 GPT 和 BERT 风格模型，所有实验都在最后 3 个变压器块中编辑 MLP 权重（共 6 个权重矩阵）。对于 BART 和 T5 风格模型，所有实验都在编码器和解码器的最后 2 个变压器块中编辑 MLP 权重（共 8 个权重矩阵）。我们发现，与编辑注意力层相比，编辑 MLP 层通常能提供更好的编辑性能（跨算法）。与过去的工作（De Cao 等人，2021 年）一致，所有报告的性能数据均基于验证集。对于所有算法，如果验证损失 $L = c_{edit}L_e + L_{loc}$ 在 500 个验证示例子集上的 20000 个步骤中没有减少，我们会提前结束训练，最大训练步数为 500,000 步。我们在所有实验中都使用了 10 个批次（梯度累积）和 0 个种子。表 7 和表 8 显示了实验中使用的每个数据集的示例。

C.1 超参数

微调。微调基线使用与模型相关的学习率，我们发现这对实现良好的微调性能非常重要；使用过大的学习率会导致局部性增强（模型退化加剧），而过小的学习率会导致编辑缓慢。我们对 GPT-Neo 和 GPT-J 模型使用 $5e-6$ 的编辑学习率，对 T5 模型使用 $1e-4$ 的编辑学习率，对较小的模型使用 $1e-6$ 的编辑学习率，目的是在不到 100 个微调步骤内完成编辑（如 De Cao 等人 (2021 年)）。对于微调 + KL 约束基线，我们在损失 $c_{edit}L_e + L_{loc}$ 上进行微调，使用的 c_{edit} 小于所学算法（除 GPT-J 外，所有模型均为 $1e-2$ ，GPT-J 需要 $1e-3$ ）。 c_{edit} 值越大，位置损失越小。为了计算 L_{loc} ，我们使用批量在每个时间步长内，从完整编辑训练集 D^{loc} 中获取一个新示例 x_{loc} 的大小。

ENN。我们使用的初始内环学习率为 $1e-2$ ，但允许在外环中学习这个值，我们发现这比 Sinitin 等人 (2020 年) 的固定内环学习率版本性能更好。在所有实验中，ENN 都会在训练过程中微调所有模型参数（即使我们只编辑最后几层）。出于计算原因，我们只使用了一个内循环更新步骤，这与 Sinitin 等人 (2020 年) 用于较小模型的多步骤版本不同。我们的编辑损失也是 Sinitin 等人 (2020) 所用编辑损失的略微简化，即

$$l_e(\vartheta) = -\log p_\vartheta(y_e | x_e, \vartheta) + \max_{y_i} \log p_\vartheta(y_i | x_{(e)}, \vartheta) \quad (6)$$

这个损耗的第一项就是我们在工作中使用的编辑损耗；第二项主要是为了在编辑成功时提供 $l_e(\vartheta) \leq 0$ 的属性，以便停止迭代编辑过程。不过，在这项工作中，由于我们只使用了单梯度编辑步骤来实现

在本文中，由于我们只对 ENN 采用了单步梯度编辑，因此这一特性并不那么重要，而第二项只相当于额外强调要特别压低最大的不正确对数（第一项已经隐含地做到了这一点）。

KE 我们使用 De Cao 等人（2021 年）提供的 KE 实现，该实现可在 <https://github.com/nicola-decao/KnowledgeEditor> 上找到，但为了与其他算法保持一致，对 KL 约束的计算略有改动（见下文）。我们使用的学习率为 $1e-5$ 。

C.2 计算局部性约束

计算编辑前模型和编辑后模型之间真正的 KL-发散 ($KL(p_{(g)}(-/x_{loc}) || p_{(g')}(-/x_{loc})$)，对于超过几个标记的模型输出来说，很快就会变得难以计算，这就需要对可能的答案进行边际化。⁶ 对于 seq2seq 问答问题，我们只在答案 y_{loc} 的标记处评估 KL 发散，得出 $KL^{(seq2seq)}(\theta, \theta') =$

$$\sum_{i=1}^{|y_{loc}|} \frac{1}{|y_{loc}|} KL(p_{(g)}(-/x_{loc}, y_{loc}^{<i>}) || p_{(g')}(-/x_{loc}, y_{loc}^{<i>})), \text{ 其中 } p_{(g)}(-/x_{loc}, y_{loc}^{<i>}) \text{ 是对下一次-分布的近似值}$$

标记 y_{loc} ，给定定位输入 x_{loc} 和之前时间步的标签标记 $y_{loc}^{<i>}$ 。同样，对于 Wikitext 设置，我们定义 $KL^{(auto)}(\theta, \theta') = \frac{1}{|x_{loc}|} \sum_{i=1}^{|x_{loc}|} KL(p_{(g)}(-/x_{loc}^{<i>}) || p_{(g')}(-/x_{loc}^{<i>}))$ 对于 FEVER_{loc}

在事实检查中，我们以封闭形式计算伯努利分布之间的精确 KL-发散。

C.3 环境详情

所有运行完全在单个英伟达 RTX Titan 或 A40 GPU 上进行训练。虽然使用了 bfloat16 将最大的 T5 模型拟合到我们的 GPU 上，但没有使用梯度检查点或内存缩减优化。在全精度情况下，仅 T5-11B 模型的参数就占用了我们最大 GPU 的全部内存。在 T5-11B 上训练 MEND 和 KE（图 3 和图 4）的 VRAM 消耗量是通过将 bfloat16 的 VRAM 使用量翻倍来估算的（Wang 和 Kanwar, 2019）。虽然将半精度提高一倍可以估算 ENN 的内存消耗，但我们无法在没有数值不稳定性的情况下以半精度训练 ENN。所有模型都基于 Huggingface Transformers 实现（Wolf 等人, 2019 年），并根据 De Cao 等人（2021 年）做了一些修改。我们在所有实验中都使用了 PyTorch（Paszke 等人, 2019 年），特别是使用 Higher 库（Grefenstette 等人, 2019 年）来实现 ENN 中的双层优化以及所有算法的模型编辑内循环。

C.4 数据集构建与示例

构建数据集的目的是提供成对的编辑输入 x_e 和可信的编辑标签 $y_{(e)}$ 。编辑标签并不一定是 "正确" 的标签；我们的目标是提供我们希望在测试过程中看到的数据类型的真实实例。例如，我们的数据集可能会有一个样本，如 x_e

= 厄休拉-勒奎恩出生在哪里？然后 y_e = 埃塞俄比亚奥罗米亚州亚的斯亚贝巴，尽管厄休拉-勒奎恩出生在美国加利福尼亚州伯克利。不过，这个虚构的例子仍然是对我们的模型执行 "更改一个人的出生地" 这种一般类型编辑能力的有益评估。对于 zsRE 问题解答数据集，De Cao 等人（2021 年）使用根据问题解答任务微调的 BART 模型的最高预测值，并通过人工过滤，以这种方式生成了虚构的 y_e 。在实践中，这样生成的替代编辑标签是可信的，其类型与原始标签相匹配。对于 FEVER 事实检查，标签只有两种选择，我们以相同的概率对编辑目标 1 和 0 进行采样。对于维基文本的生成，我们使用 distilGPT-2 模型来为给定的维基文本前缀生成可信的 10 个标记后缀，其动机与 zsRE 类似，都是为了提供与我们实际应用中的编辑类型结构相同的编辑目标，即使它们并不总是符合事实。在使用事实标签对 MEND 进行定性评估以纠正基础模型的真实错误时，我们发现 MEND 的表现非常可靠，这表明这些标签生成器为 "真实" 模型编辑提供了合理的代理。

⁶ 我们之所以这样选择，是因为模型的预测分布类似于局部样本分布（因为局部样本来自模型最初训练的数据集）。

(ICLH) 上发表
虽然这不如使用模型本身样本进行真正的蒙特卡罗估计更有原则性，但它降低了训练的计算要求，而且更容易实现；大多数模型的缩减率普遍较低，这表明这种近似方法在实践中仍能提供良好的局部性约束。

(ICLR) 上发表

$x_{\text{ce}}, y_{\text{ce}}$	萨普朗曾被认为是 2007 年国家安全委员会领导人颂提-汶亚拉特卡林 (Sonthi Boonyaratkalin) 强制退休后领导军队和军政府的最有力竞争者之一。然而, 2007 年 9 月他被降职为国防部常务副秘书长, 而他的竞争对手阿努蓬-保钦达将军则被提升为 副总司令 。后来, 他的职位被 取代
x_{loc}	1663 年, 苏格兰数学家詹姆斯-格雷戈里 (James Gregory) 在他的《Optica Promota》一书中提出, 在地球表面间距较大的点上观测水星凌日, 可以利用三角测量法计算出太阳视差, 进而计算出公历单位。意识到这一点后, 年轻的埃德蒙-哈雷 (Edmond Halley) 于公元 1677 年 10 月 28 日从圣赫勒拿岛观测到了水星凌日, 但他失望地发现, 只有兰开夏郡伯恩利的理查德-汤利 (Richard Towneley) 对这一事件进行了另一次精确观测, 而阿维尼翁的加莱 (Gallet) 只是记录了这一事件的发生。哈雷对由此计算出的 45" 太阳视差的准确性并不满意。
$x'_{\text{ce}}, y'_{\text{ce}}$	然而, 2007 年 9 月, 他被降职为英国科学院副常务秘书。 然而, 2007 年 9 月, 他被降职为国防部副常务秘书, 而他的竞争对手阿努蓬-保钦达将军则被提升为副检察长。 副总检察长。后来, 他被取代

表 8: 维基文本编辑数据集的训练集示例。粗体文字对应编辑标签 y_{ce} 和 y'_{ce} 。位置示例 x_{loc} 用于限制编辑前和编辑后模型的预测分布在序列中的每个标记处都相似。

D MLP 的 RANK-1 梯度

在 MLP 和批量大小为 1 的简化情况下, 我们描述损失 L 相对于第 l 层权重矩阵 w_A 的秩-1 梯度。我们将第 l 层的输入定义为 u_A , 将第 l 层 + 1 的预激活输入定义为 $z_{(A) (+1)} = w_{(A)} u_{(A)}$ 。我们将 $\delta_{(A) (+1)}$ 定义为 L 相对于 $z_{(A) (+1)}$ 的梯度 (我们假设 $\delta_{(A) (+1)}$ 是预先计算的, 是标准反向传播的结果)。我们将证明损失 L 相对于 w_A 的梯度等于 $\delta_{(A) (+1)} u^{(T)}$ 。

根据链式法则, 损失相对于权重 w^{ij} 的导数等于

$$\frac{\partial L}{\partial w_A^{ij}} = \sum_k \frac{\partial L}{\partial z_{A+1}^k} \frac{\partial z_{A+1}^k}{\partial w_A^{ij}} = \frac{\partial L}{\partial z_{A+1}^i} \frac{\partial z_{A+1}^i}{\partial w_A^{ij}} \quad (7)$$

L 相对于下一层预激活 z 的导数乘积

与导数

的下一层预激活 z

与 w_{ij} 的关系。第二个等式是由于

$$\frac{\partial z_{A+1}^k}{\partial w_A^{ij}} = 0 \text{ for } k \neq i. \text{ 注意到 } z_{A+1}^i = \sum_{j,A} u^{(j)} w_{ij}, \text{ 我们可以用简单的 } u^{(j)} \text{ 替换 } \frac{\partial z_{A+1}^i}{\partial w_A^{ij}} \text{ 中的 } u^i$$

等式 7。此外, 我们还定义了 δ_{A+1} 正好是 $(\frac{\partial L}{\partial z_{A+1}})$ 。进行这两种替换后, 我们得到

$$\frac{\partial L}{\partial w_A^{ij}} = \delta_{A+1}^i u^j \quad (8)$$

或者, 用向量符号表示, $\nabla_w L = \delta_{A+1} u^{(T)}$, 这就是我们要证明的原始特性。

E 编辑注意力参数

我们的实验编辑了大型变压器 MLP 层的权重。表 9 显示的是编辑注意力层而非 MLP 层的结果, 与编辑 MLP 层相比, 编辑注意力层通常会降低性能。为了进行比较, 我们编辑了与表 3 中主要编辑实验相同的转换器块, 但我们编辑的是每个块的查询/键/值/输出矩阵, 而不是两个 MLP 矩阵。编辑 MLP 层更有效的观察结果与过去的工作 (Geva 等人, 2021 年) 基本一致, 即 Transformer 架构中的 MLP 层将人类可解释的高级概念存储在模型的后几层, 这也是我们在最初实验中选择编辑这些层的原因。此外, 我们还假设, 编辑 MLP 层之所以能提高效率, 可能只是因为它们占了模型参数的绝大部分, 因为 MLP 的隐藏状态通常比

	维基文本生成				zsRE 问题解答			
	GPT-Neo (2.7B)		GPT-J (6B)		T5-XL (2.8B)		T5-XXL (11B)	
编辑器	ES ↑	ppl. DD ↓	ES ↑	ppl. DD ↓	ES ↑	acc. DD ↓	ES ↑	acc. DD ↓
MEND-attention	0.73	0.068	0.54	0.122	0.63	0.001	0.78	<0.001
MEND-mlp (表 3)	0.81	0.057	0.88	0.031	0.88	0.001	0.89	<0.001

表 9: 编辑表 3 中模型的注意力矩阵而不是 MLP/前馈参数。就生成模型的缩减和编辑成功率以及 T5 seq2seq 模型的编辑成功率而言, 编辑注意力参数始终会降低编辑性能。

输入	编辑前输出	编辑目标	编辑后输出
1a: 谁是美国总统?	唐纳德·特朗普 C	乔·拜登	乔·拜登✓ 1b
: 谁是美国总统?	戴维·赖斯-艾奇逊 C -	乔·拜登 ✓	
1c: 谁是法国总统?	埃马纽埃尔·马克龙 -✓		埃马纽埃尔·马克龙 ✓
2a: 谁设计了哈利法塔?	英国建筑师 赫伯特·贝克 C	斯基德莫尔、欧文斯和梅里尔公司	斯基德莫尔、欧文斯和梅里尔事务所 ✓
2b: 埃菲尔铁塔是谁设计的?	亚历山大·古斯塔夫埃菲尔铁塔 ✓	-	亚历山大·古斯塔夫-埃菲尔 ✓
2c: 帝国大厦是谁设计的?	什里夫、兰姆和哈蒙公司 ✓	-	什里夫、兰姆和哈蒙 ✓
2d: 悉尼歌剧院是谁设计的? Jørn Oberg Utzon -✓			Jørn Oberg Utzon *✓
2e: 设计哈利法塔的是哪家公司?	McKim, Mead & White C	-	斯基德莫尔、欧文斯和梅里尔事务所 ✓
2f: 哈利法塔是哪家公司设计的?	朱美拉集团 C	-	朱美拉集团 C
3a: Astra 是哪家汽车公司生产的?	马恒达 C	欧宝	欧宝✓
3b: 哪个汽车公司生产野马?	福特 -✓		福特✓
3c: 哪个汽车公司生产 Model S?	特斯拉汽车公司 -✓		特斯拉✓
3d: 哪个汽车公司生产牧马人? 吉普车 -✓		Jeep✓ 3e: 哪个汽车公	
司生产 F-150?	福特 - 欧宝 C✓		欧宝 C
3f: 哪个汽车公司生产高尔夫?	大众汽车公司✓	- 欧宝 C	欧宝 C
4a: 哪个艺术家录制了《颤栗》?	麦当娜 C	迈克尔·杰克逊	迈克尔·杰克逊 ✓
4b: 哪位艺术家录制了《月之暗面》?	平克·弗洛伊德 -✓		平克·弗洛伊德 ✓
4c: 哪位艺术家录制了《乱水上的桥》?	西蒙和加丰克尔 -✓		西蒙与加丰克尔 ✓
4d: 哪位艺术家录制了《加州旅馆》?	唐·亨利?	-	唐·亨利?
4e: 哪个乐队录制了《Back in Black》?	AC/DC -✓		迈克尔·杰克逊 C

表 10: 使用 MEND 编辑根据自然问题 (NQ; Kwiatkowski 等人 (2019)) 微调的 770M 参数 T5-large 模型的其他示例。

示例 2e 显示了正确的泛化行为; 示例 2f 显示了泛化不足的实例; 示例 3e、3f 和 4e 显示了泛化过度的实例。

*虽然符号 ø 并未正确生成, 但我们仍将其视为正确答案 (Jørn Oberg Utzon 才是正确答案)。

编辑器	发烧		zsRE		zsRE-硬		维基文本	
	BERT-基地		BART- base		BART-基础		蒸馏 GPT-2	
	ES $\uparrow$	acc. DD $\downarrow$	ES $\uparrow$	acc. DD $\downarrow$	ES $\uparrow$	ppl. DD $\downarrow$	ES $\uparrow$	ppl. DD $\downarrow$
MEND	>0.99	<0.001	0.98	0.002	0.66	<0.001	0.86	0.225
缓存 (g) [*]	0.96	<0.001	>0.99	0.002	0.32	0.002	0.001	0.211
缓存 (g) ^{2g}	0.70	<0.001	0.70	<0.001	-	-	<0.001	0.037
缓存 (g) ^{2g}	>0.99	0.250	1.00	0.220	-	-	0.002	2.770

表 11: 将 MEND 与基于缓存的编辑方法进行比较。为了便于比较, 缓存隐藏状态相似性阈值 g^* 是与 MEND 相似的缩减阈值。我们发现 FEVER、zsRE 和 Wikitext 的 g^* 分别为 6.5、3、2.5。**上半部分**。缓存对 zsRE 的性能稍好, 对 FEVER 的性能稍差, 而对 Wikitext 编辑则完全失败, 这可能是由于 Wikitext 数据中的上下文更长、更复杂。**下半部分**缓存对所选阈值比较敏感, 需要针对每个新任务分别进行调整。

F MEND 的其他定性示例

我们在表 10 中提供了使用 MEND 编辑更大的 770M 参数 T5-large 模型 (罗伯茨等人, 2020 年) 的其他定性示例。这些示例包括一个**未充分概括**的实例, 即编辑示例的输出被正确编辑, 但编辑示例的等价邻域中的其他示例没有变化 (见表 10 中的 2f)。此外, 我们还强调了**过度泛化**的失败案例, 在这种情况下, 模型对表面相似但语义不同的输入的编辑后输出也是编辑目标; 例如表 10 中的 3e、3f 和 4e。减少模型编辑器的这些失败情况 (确保) 是未来工作的一个重要优先事项、

G 通过缓存进行编辑

另一种简单的编辑方法可能是缓存编辑示例 x_e 的最终层隐藏状态 $z_{(e)}$ (序列长度的平均值) 和相应编辑标签 $y_{(e)}$ 的标记。编辑完成后, 如果模型接收到一个新的输入 x , 其最终层隐藏状态 z 接近于 z_e (即 $\|z - z_e\|_2 < \epsilon$) , 那么模型就会输出 y_e , 而不是正常预测。在这里, 我们展示了这种方法对于输入较简单的编辑问题 (zsRE 问题解答、FEVER 事实核查) 是有效的, 在这些问题中, 输入通常是短小、简单的短语, 只有一个主语、一个关系和一个宾语, 但在维基文本编辑问题上却完全失败了, 在维基文本编辑问题中, 上下文通常长达 10 倍, 不同的段落包含大量无关文本和 "干扰" 信息。结果见表 11。其中包括 "最佳" 阈值 ϵ^* (与 MEND 达到相似缩减的阈值), 以及使用 $2\epsilon^*$ 和 $(1)\epsilon^*$ 的结果。我们观察到, 缓存方法对阈值超参数相当敏感, 对某项任务有效的阈值可能对其他任务无效。

对于 zsRE 问题解答, z 是作为问题标记的平均隐藏状态来计算的; 对于 FEVER 事实检查, z 是事实陈述标记的平均隐藏状态。对于生成式建模, 在预测时间步长为 t 的标记时, 我们将 z_t 计算为所有之前见过的标记的平均隐藏状态 $< t$ 。为了计算缓存方法的困惑度, 我们输出了与 y_e 相对应的一热对数。我们尝试了按不同系数缩放一热对数, 但发现缩放 1 效果很好; 缩放相当于改变模型对其编辑预测的信心, 但不会改变预测本身或编辑成功率。